

MoveIt!: An Introduction

Sachin Chitta

Abstract MoveIt! is state of the art software for mobile manipulation, incorporating the latest advances in motion planning, manipulation, 3D perception, kinematics, control and navigation. It provides an easy-to-use platform for developing advanced robotics applications, evaluating new robot designs and building integrated robotics products for industrial, commercial, R&D and other domains. MoveIt! is the most widely used open-source software for manipulation and has been used on over 65 different robots. This tutorial is intended for both new and advanced users: it will teach new users how to integrate MoveIt! with their robots while advanced users will also be able to get information on features that they may not be familiar with.

1 Introduction

Robotics has undergone a transformational change over the last decade. The advent of new open-source frameworks like ROS and MoveIt! has made robotics more accessible to new users, both in research and consumer applications. In particular, ROS has revolutionized the *developers* community, providing it with a set of tools, infrastructure and best practices to build new applications and robots (like the Baxter research robot). A key pillar of the ROS effort is the notion of *not re-inventing the wheel* by providing easy to use libraries for different capabilities like navigation, manipulation, control (and more).

MoveIt! provides the core functionality for manipulation in ROS. MoveIt! builds on multiple pillars:

- **A library of capabilities:** MoveIt! provides a library of robotic capabilities for manipulation, motion planning, control and mobile manipulation.
- **A strong community:** A strong community of users and developers that help in maintaining and extending MoveIt! to new applications.

S. Chitta (✉)
Kinema Systems Inc., Menlo Park, CA 94025, USA
e-mail: robot.moveit@gmail.com
URL: <http://moveit.ros.org>

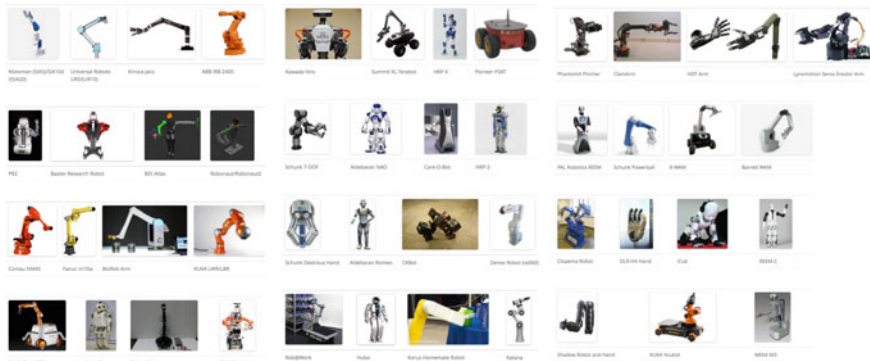


Fig. 1 Robots using MoveIt!

- **Tools:** A set of tools that allow new users to integrate MoveIt! with their robots and advanced users to develop new applications.

Figure 1 shows a list of robots that MoveIt! has been used with. The robots range from industrial robots from all the leading vendors and research robots from all over the world. The robots include single arm, dual-armed robots, mobile manipulation systems, and humanoid robots. MoveIt! has been used in applications ranging from search and rescue (the DARPA robotics challenge), unstructured autonomous pick and place (with industrial robots like the UR5), mobile manipulation (with the PR2 and other robots), process tasks like painting and welding, with the (simulated) Robonaut robot for target applications in the space station. MoveIt! has been used or will be used by teams in the DARPA Robotics Challenge, the ROS-Industrial Consortium, the upcoming Amazon Picking Challenge, the NASA sample retrieval challenge.

2 A Brief History

MoveIt! evolved from the Arm Navigation framework [1, 2] in ROS. The Arm Navigation framework was developed after the development of the base navigation stack in ROS to provide the same functionality that was now available for base navigation in ROS. It combined kinematics, motion planning, 3D perception and an interface to control to provide the base functionality of *moving an arm* in unstructured environments. The central node in the Arm Navigation framework, called *move_arm*, was designed to be robot agnostic, i.e. usable with any robot. It connected to several other nodes, for kinematics, motion planning, 3D perception and other capabilities, to generate collision-free trajectories for robot arms. The Arm Navigation framework was further combined with the ROS grasping pipeline to create, for the first time, a general grasping and manipulation framework that could (and was) ported onto several different robots with different kinematics and grippers.

The Arm Navigation framework had several shortcomings. Each capability in the framework was designed as a separate ROS node. This required sharing data (particularly environment data) across several processes. The need for synchronization between the nodes led to several issues: (a) a mis-match of state between separate nodes often resulted in motion plans that were invalid, (b) communication bottlenecks because of the need to send expensive 3D data to several different nodes and (c) difficulty in extending the types of services offered by *move_arm* since it required changing the structure of the node itself. MoveIt! was designed to address all of these issues.

3 MoveIt! Architecture

The architecture of MoveIt! is shown in Fig. 2. The central node in MoveIt! is called *move_group*. It is intended to be light-weight, managing different capabilities and integrating kinematics, motion planning and perception. It uses a plugin-based architecture (adopted from ROS)—dramatically improving MoveIt!’s extensibility when compared to Arm Navigation. The plugin architecture allows users to add and share *capabilities* easily, e.g. a new implementation of pick and place or motion planning. The use of plugins is a central feature of MoveIt! and differentiates it from Arm Navigation.

Users can access the actions and services provided by *move_group* in one of three ways:

- **In C++:** using the *move_group_interface* package that provides an easy to setup interface to *move_group* using a C++ API. This API is primarily meant for advanced users and is useful when creating higher-level capabilities.

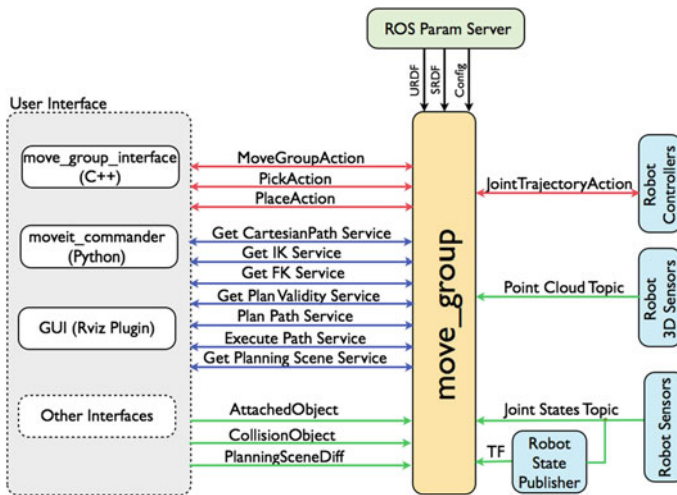


Fig. 2 MoveIt! high-level architecture

- **In Python:** using the *moveit_commander* package. This API is recommended for scripting demos and for building applications.
- **Through a GUI:** using the Motion Planning plugin to Rviz (the ROS visualizer). This API is recommended for visualization, initial interaction with robots through MoveIt! and for quick demonstrations.

One of the primary design principles behind MoveIt! is to expose an easy to use API for beginners to use while retaining access to the entire underlying API for more advanced users. MoveIt! users can access any part of the functionality directly if desired, allowing custom users to modify and architect their own applications. MoveIt! builds on several component technologies, each of which we will describe in brief detail.

3.1 Collision Checking

MoveIt! relies on the FCL [3] package for native collision checking. The collision checking capabilities are implemented using a plugin architecture, allowing any collision checker to be integrated with MoveIt!. FCL provides a state of the art implementation of collision checking, including the ability to do continuous collision checking. Collision checking is often the most expensive part of motion planning, consuming almost 80–90 % of the time for generating a motion plan. The use of an ALLOWED COLLISION MATRIX allows a user to specify which pairs of bodies do not need to be checked against each other, saving significant time. The ALLOWED COLLISION MATRIX is automatically configured by the MoveIt! Setup Assistant but can also be modified online by the user.

3.2 Kinematics

MoveIt! utilizes a plugin-based architecture for solving inverse kinematics while providing a native implementation of forward kinematics. Natively, MoveIt! uses a numerical solver for inverse kinematics for any robot. Users are free to add their own custom solvers, in particular analytic solvers are much faster than the native solver. Examples of analytic solvers that are integrated with MoveIt! include the solver for the PR2 robot. A popular plugin-based solver for MoveIt! is based on IKFast [4] and offers analytic solvers for industrial arms that are generated (in code).

3.3 Motion Planning

MoveIt! works with motion planners through a plugin interface. This allows MoveIt! to communicate with and use different motion planners from multiple libraries,

making MoveIt! easily extensible. The interface to the motion planners is through a ROS Action or service (offered by the `move_group` node). The default motion planners for `move_group` are configured using the MoveIt! Setup Assistant. OMPL (Open Motion Planning Library) is an open-source motion planning library that primarily implements randomized motion planners. MoveIt! integrates directly with OMPL and uses the motion planners from that library as its primary/default set of planners. The planners in OMPL are abstract; i.e. OMPL has no concept of a robot. Instead, MoveIt! configures OMPL and provides the back-end for OMPL to work with problems in Robotics.

3.4 Planning Scene

The planning scene is used to represent the world around the robot and also stores the state of the robot itself. It is maintained by the planning scene monitor inside the `move_group` node. The planning scene monitor listens to:

- **Robot State Information:** on the `joint_states` topic and using transform information from the ROS TF transform tree.
- **Sensor Information:** using a world geometry monitor that integrates 3D occupancy information and other object information.
- **World Geometry Information:** from user input or other sources, e.g. from an object recognition service.

The planning scene interface provides the primary interface for users to modify the state of the world that the robot operates in.

3.5 3D Perception

3D perception in MoveIt! is handled by the occupancy map monitor. The Occupancy map monitor uses an Octomap to maintain the occupancy map of the environment. The Octomap can actually encode probabilistic information about individual cells although this information is not currently used in MoveIt!. The Octomap can directly be passed into FCL, the collision checking library that MoveIt! uses. Input to the occupancy map monitor is from depth images, e.g. from an ASUS Xtion Pro Sensor or the Kinect 2 sensor. The depth image occupancy map updater includes its own self-filter, i.e. it will remove visible parts of the robot from the depth map. It uses current information about the robot (the robot state) to carry out this operation. Figure 3 shows the architecture corresponding to the 3D perception components in MoveIt!.

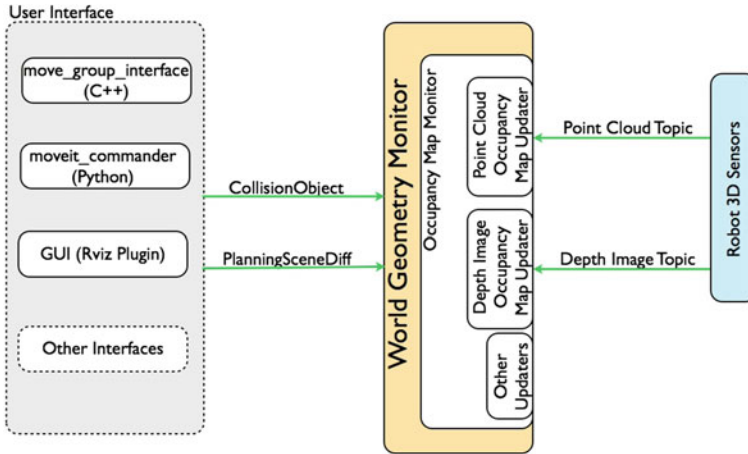


Fig. 3 The 3D perception pipeline in MoveIt!: architecture

3.6 Trajectory Processing

MoveIt! includes a trajectory processing component. Motion planners will typically only generate paths, i.e. there is no timing information associated with the paths. MoveIt! includes trajectory processing routines that can work on these paths and generate trajectories that are properly time-parameterized accounting for the maximum velocity and acceleration limits imposed on individual joints. These limits are read from a separate file specified for each robot.

3.7 Using This Tutorial

MoveIt! is a large package and it is impossible to cover it in its entirety in a book chapter. This document serves as a reference for the tutorial that users can use but must be used in conjunction with the online documentation on the [MoveIt! website](#). The online resource will remain the most up to date source of information on MoveIt!. This paper will introduce the most important concepts in MoveIt! and also provide helpful hints for new users. We assume that the user is already familiar with ROS. Readers should go through the [ROS Tutorials](#)—in particular, they should learn about ROS topics, services, using the ROS parameter server, ROS actions, the ROS build system and the ROS transform infrastructure (TF).

The example URDFs and MoveIt! config packages used in this tutorial for the Fanuc M10ia robot can be found in the [examples repository](#).

3.8 *Installing MoveIt!*

MoveIt! can easily be installed on a Ubuntu 14.04 distribution using ROS Indigo. The most updated instructions for installing MoveIt! can be found on the [MoveIt! installation page](#). It is recommended that most users follow the instructions for installing from binaries. There are three steps to installing MoveIt!:

1. Install ROS—follow the latest instructions on the [ROS installation page](#).
2. Install MoveIt!:

```
sudo apt-get install ros-indigo-moveit-full
```

3. Setup your environment:

```
source /opt/ros/indigo/setup.bash
```

4 Starting with MoveIt!: The Setup Assistant

The first step in working with MoveIt! is to use the MoveIt! Setup Assistant.¹ The setup assistant is designed to allow users to import new robots and create a MoveIt! package for interacting, visualizing and simulating their robot (and associated workcell). The primary function of the setup assistant is to generate a Semantic Robot Description Format (SRDF) file for the robot. It also generates a set of files that allow the user to start a visualized demonstration of the robot instantly. We will not describe the Setup Assistant in detail (the latest instructions can always be found on the MoveIt! website [5]). We will instead focus on the parts of the process that creates the most confusion for new users.

4.1 *Start*

To start the setup assistant:

```
roslaunch moveit_setup_assistant moveit_setup_assistant
```

This will bring up a startup screen with two choices: Create New MoveIt! Configuration Package or Edit Existing MoveIt! Configuration Package. Users should select Create New MoveIt! Configuration Package for any new robot or workcell (even if the robots in the workcells already have their own configuration package). Figure 4 illustrates this for

¹This tutorial assumes that the user is using ROS Indigo on a Ubuntu 14.04 distribution.

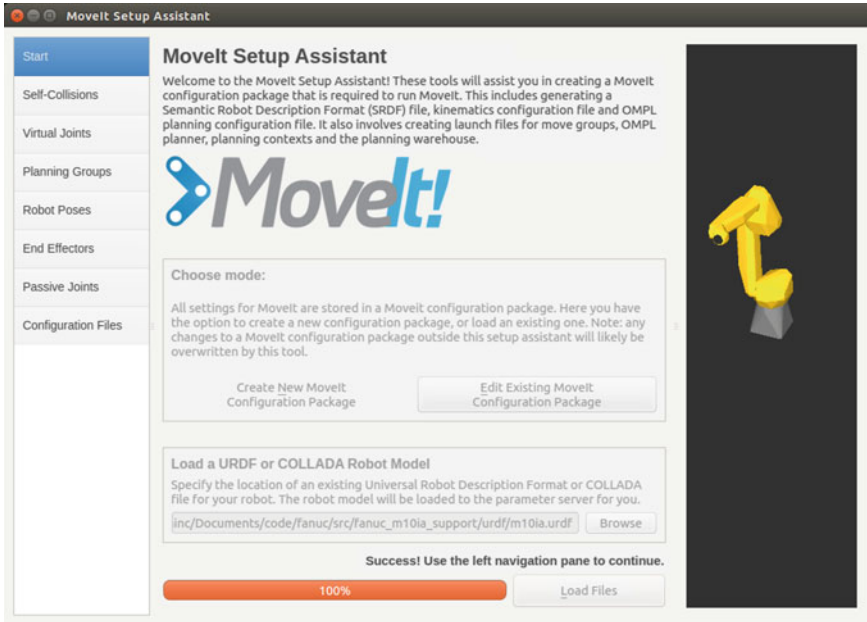


Fig. 4 Loading a Robot into the Setup Assistant

a Fanuc M10ia robot. Note that users can select either a URDF file or a *xacro* file (often used to put together multiple robots).

The Setup Assistant is also capable of editing an existing configuration. The primary reason to edit an existing configuration is to regenerate the Allowed Collision Matrix (ACM). This matrix needs to be re-generated when any of the following happens:

- The geometric description of your robot (URDF) has changed—i.e., the mesh representation being used for the robot has changed. Note here that the collision mesh representation is the key component of the URDF that MoveIt! uses. Changing the visual description of the robot while keeping the collision representation unchanged will not require the MoveIt! Setup Assistant to be run again.
- The joint limits specified for the robot have changed—this changes the limits that the Setup Assistant uses in sampling states for the Allowed Collision Matrix (ACM). Failing to run the Setup Assistant again may result in a state where the robot is allowed to move into configurations where it could be in collision with itself or with other parts of the environment.

4.2 Generating the Self-Collision Matrix

The key choice in generating the self-collision matrix is the number of random samples to be generated. Using a higher number results in more samples being generated but also slows down the process of generating the MoveIt! config package. Selecting a lower number implies that fewer samples are generated and there is a possibility that some collision checks may be wrongly disabled. We have found in practice, that generating at least 10,000 samples (the default value) is a good practice. Figure 5 shows what you should expect to see at the end of this step (Remember to press the SAVE button!).

4.3 Add Virtual Joints

Virtual joints are sometimes required to specify where the robot is in the *world*. A virtual joint could be related to the motion of a mobile base or it could be fixed, e.g. for an industrial robot bolted to the ground. Virtual joints are not always required—you can work with the default URDF model of the robot for most robots. If you do add a virtual joint, remember that there has to be a source of transform information for it (e.g. a localization module for a mobile base or a TF static transform publisher

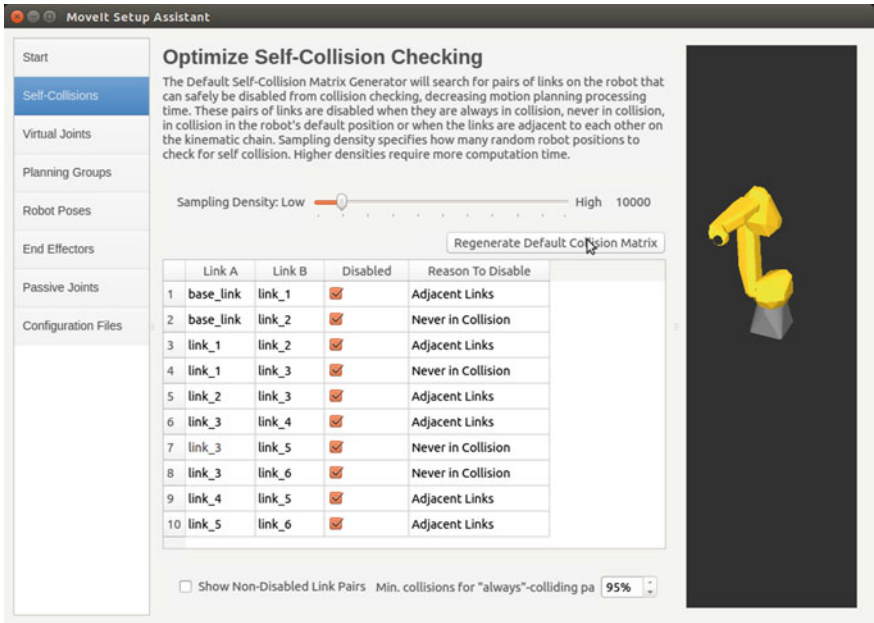


Fig. 5 Generating the self-collision matrix

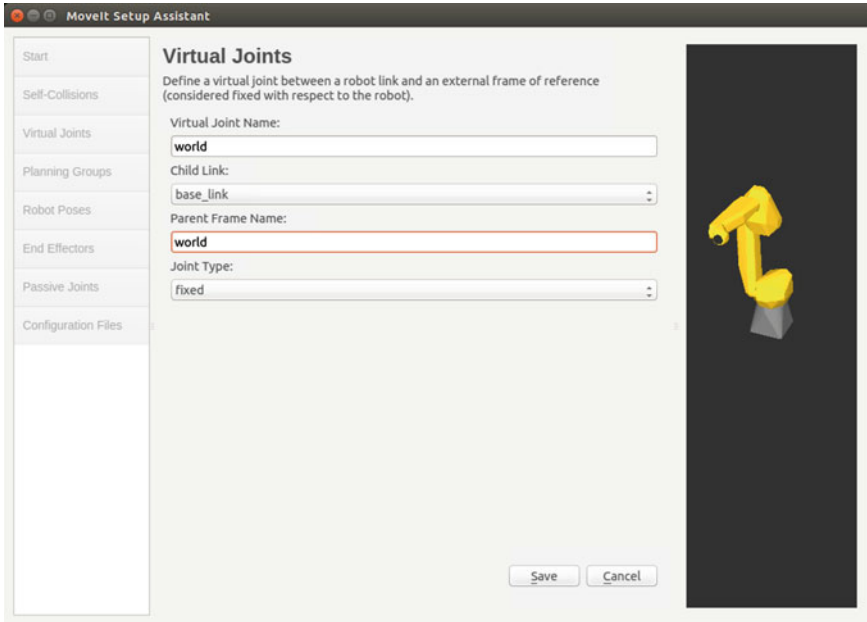


Fig. 6 Adding virtual joints

for a fixed robot). Figure 6 illustrates the process of adding a *fixed* joint that attaches the robot to the world.

4.4 Planning Groups

Planning groups bring together, semantically, different parts of the robot into a group, e.g. an arm or a leg. The definition of groups is the primary function of the SRDF. In the future, it is hoped that this information will move directly into the URDF. Groups are typically defined by grouping a set of joints together. Every child link of the joints is now a member of the group. Groups can also be defined as a chain by specifying the first link and the last link in the chain—this is more convenient when defining an arm or a leg.

In defining a group, you also have the opportunity to define a *kinematic solver* for the group (note that this choice is optional). The default kinematic solver that is always available for a group is the MoveIt! KDL Kinematics solver built around the Kinematics Dynamics Library package (KDL). This solver will only work with *chains*. It automatically checks (at startup) whether the group it is configured for is a chain or a disjoint collection of joints. Custom kinematics solvers can also be integrated into MoveIt! using a plugin architecture and will show up in the list of

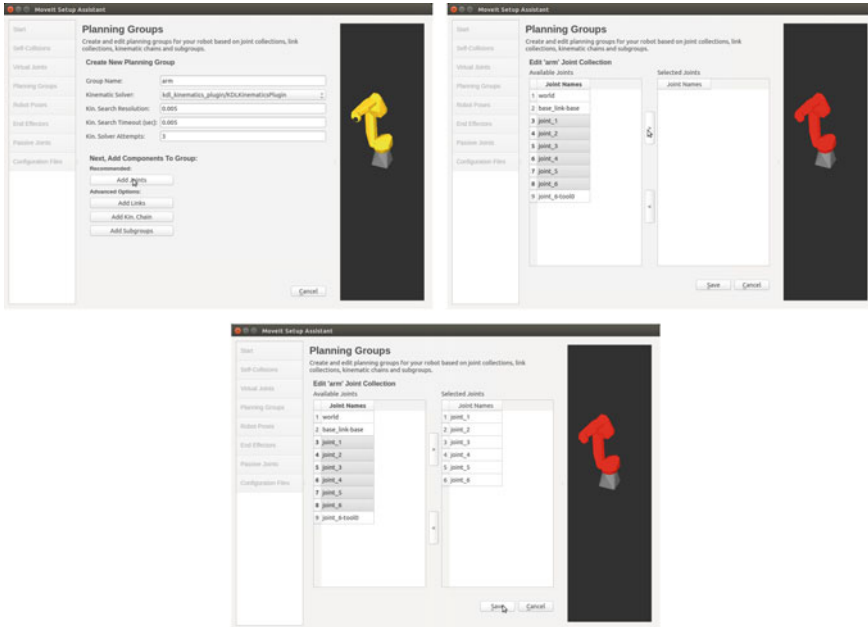


Fig. 7 Adding planning groups

choices for choosing a kinematics solver. Note that you may (and should) elect not to initialize a kinematics solver for certain groups (e.g. a parallel jaw gripper).

Figure 7 show an example where a Fanuc robot arm is configured to have a group that represents its six joints. The joints are added to the group using the “Add Joints” button (which is the recommended button). You can also define a group using just a link, e.g. to define an end-effector for the Fanuc M10ia robot, you would use the *tool0* link to define an end-effector group.

4.5 Robot Poses

The user may also add fixed poses of the robot into the SRDF. These poses are often used to describe configurations that are useful in different situations, e.g. a home position. These poses are then easily accessible using the internal C++ API of MoveIt!. Figure 8 shows a pose defined for the Fanuc M10ia. Note that these poses are user-defined and do not correspond to a native zero or home pose for the robot.

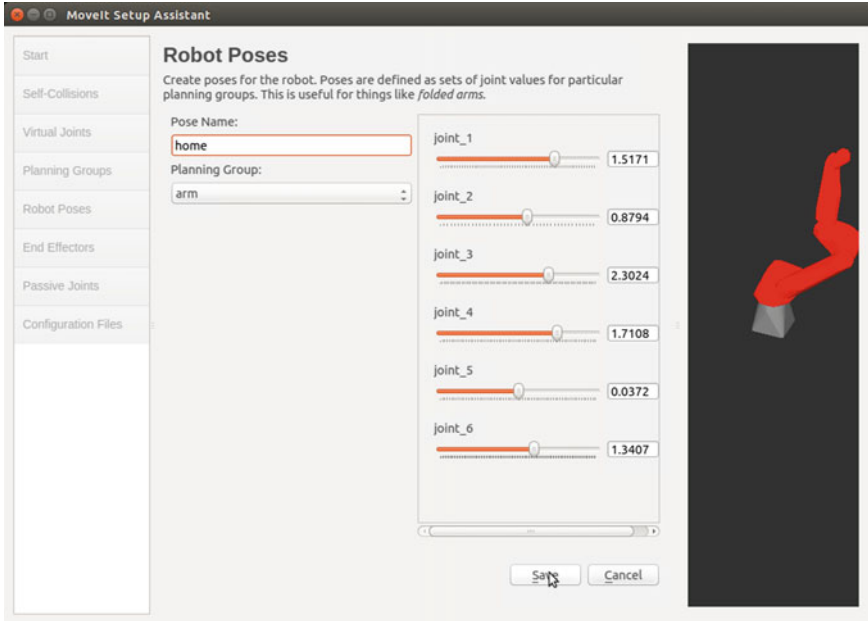


Fig. 8 Adding robot poses

4.6 Passive Joints

Certain joints in the robot can be designated as *passive joints*. This allows the various components of MoveIt! to know that such joints cannot be used for planning or control.

4.7 Adding End-Effectors (Optional)

Certain groups in the robot can be designated as *end-effectors*. This allows users to interact through these groups using the Rviz interface. Figure 9 shows the *gripper* group being designated as an end-effector.

4.8 Configuration Files

The last step in the MoveIt! Setup Assistant is to generate the configuration files that MoveIt! will use (Fig. 10). Note that it is convention to name the generated MoveIt! config package as `robot_name_moveit_config`. E.g. for the Fanuc robot used in our

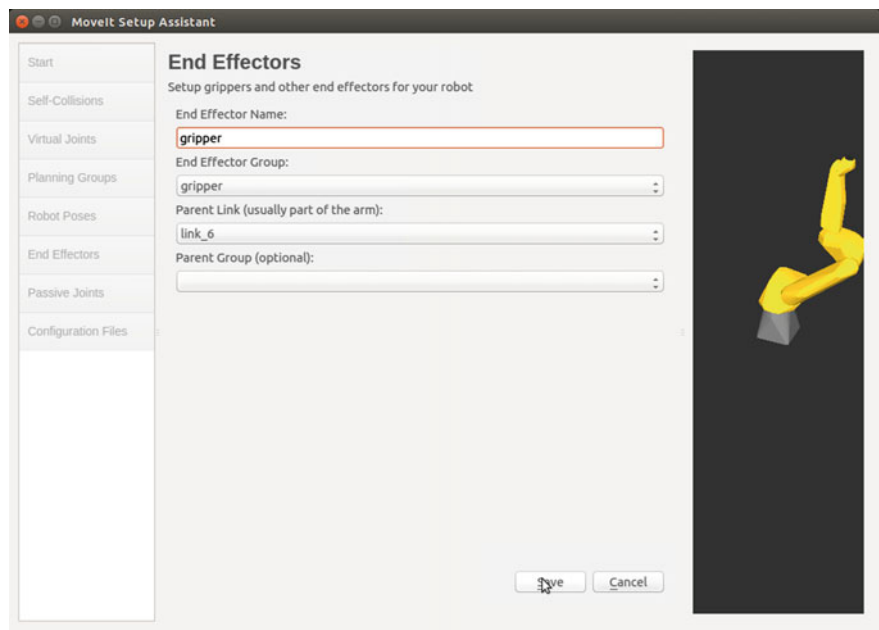


Fig. 9 Adding end-effectors

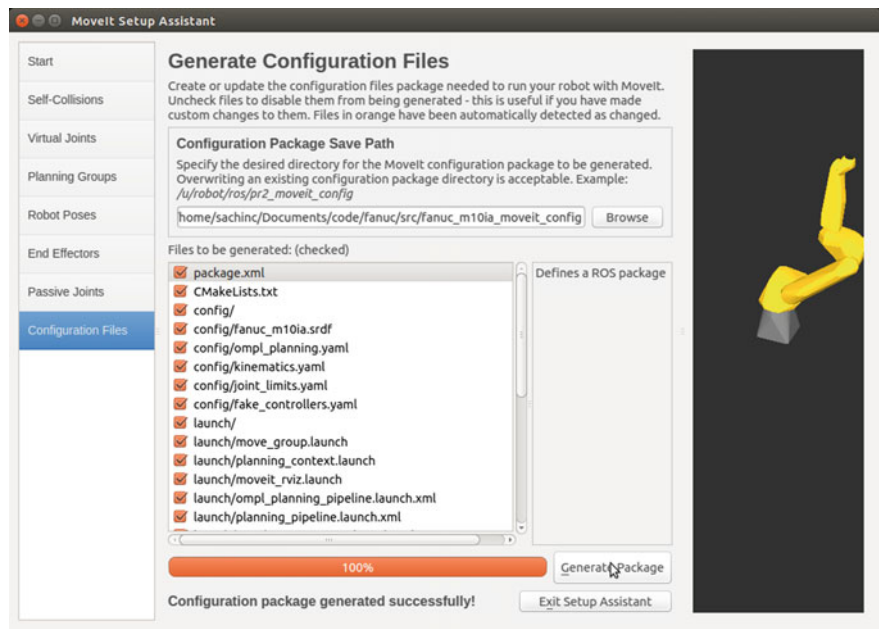


Fig. 10 Generating configuration files

example, we would name the package `fanuc_m10ia_moveit_config`. Now, running the initial demonstration is easy:

```
roslaunch <moveit_config_package_name> demo.launch
```

This entire process is better illustrated in the [movie accompanying this paper](#).

5 Using the Rviz Motion Planning Plugin

The Rviz motion planning plugin is the primary interface for working with MoveIt!. It allows users to create environments and plan collision-free motions. It is shown in Fig. 11. Here, using the Context tab, users can make a choice of the type of motion planner to use. The default planning library available with MoveIt! is the OMPL library—it is automatically configured by the MoveIt! Setup Assistant. The next tab (labeled PLANNING) is shown in Fig. 12. It allows users to adjust parameters for the planning process, including allowed planning time, the number of planning attempts and the speed of the desired motion (as a percentage of full speed). It also allows users to configure the start and goal states for planning—they can be configured randomly, to match the current state of a simulated (or real) robot or to a *named* state setup in the MoveIt! Setup Assistant. The PLANNING tab also allows the user to plan and execute motions for the robot.

The Scene tab allows users to add or delete objects into the workspace of the robot. Objects or workcells are typically loaded directly from CAD models, e.g. STL files.

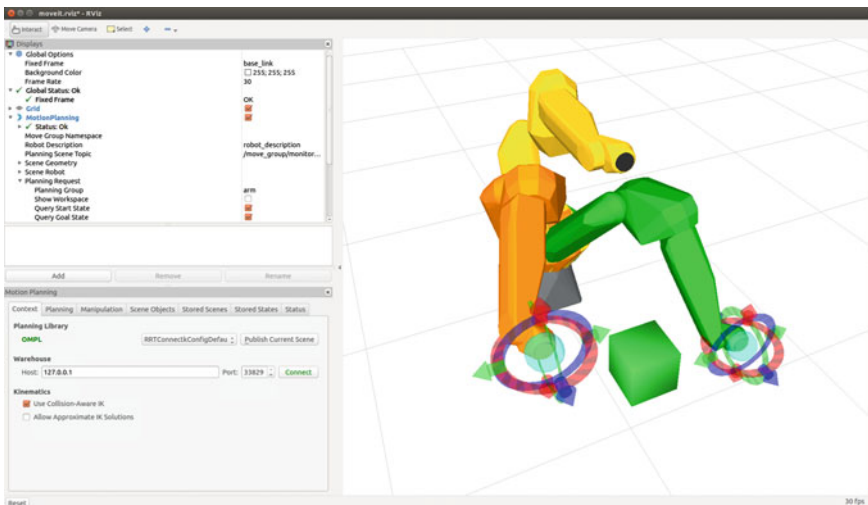


Fig. 11 MoveIt! Rviz Plugin

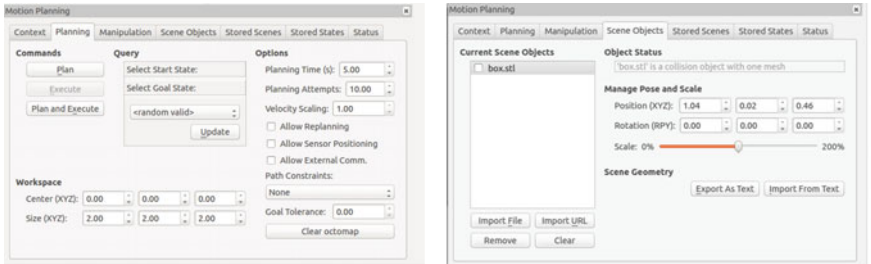
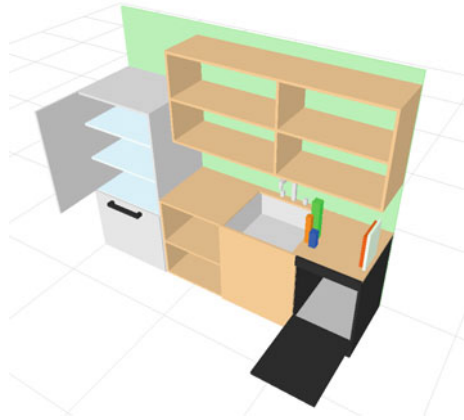


Fig. 12 MoveIt! Rviz Plugin: the planning interface (*left*) and the scene tab (*right*)

Fig. 13 MoveIt! Rviz Plugin: an imported model

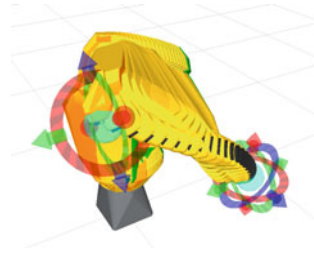


To explore this functionality, first download two files: a simple STL file representing a box and a representation of a complete *scene* for the robot to work in (in the *.scene* format). These files can be imported into the Planning Scene using the import buttons in the SCENE tab. To import STL files, use the IMPORT FILE button. To import Scene files, use the Import From Text button. Figure 13 shows a countertop model imported into MoveIt!. Once you import a model, make sure to return to the CONTEXT tab and press the Publish Current Scene button to indicate to MoveIt! that it should be working with an updated model of the environment.

5.1 Visualization and Interaction

The Rviz motion planning plugin is also the primary tool for visualization of MoveIt! plans and for interaction with MoveIt! robot models. Figure 14 shows the robot interaction models that the user can interact with. A blue ball attached to the end-effector allows the user to easily drag the end-effector around in any environment while ROS interactive markers are used for finer position and orientation control of

Fig. 14 MoveIt! Rviz Plugin: interaction and visualization. The *blue ball* allows direct position interaction with the end-effector of a robot. Path traces can be visualized in MoveIt!



the end-effector. Planned paths are visualized using the plugin and can be cycled continuously for better introspection.

5.2 Useful Hints

It is important, while using the MoveIt! Rviz Plugin, to make sure that the start state for planning always matches the current state of the robot. It is also important to set the speed to a conservative limit initially, especially with industrial robots, to make sure that the robot is moving safely in its environment. User should also check the collision model of their workcell and ensure that it is an accurate representation of their workcell.

6 The move_group_interface

The primary recommended code API to MoveIt! is through the *move_group_interface*. It provides both a C++ and Python API to the *move_group* node. The interface abstracts the ROS API to MoveIt! and makes it easier to use. The ROS API is configured primarily using *constraints*, e.g. a position constraint for the end-effector or joint constraints for the entire robot. A position constraint is typically specified using a box volume in space for the end-effector. The *move_group_interface* allows users to specify these constraints directly as a desired position, orientation or joint configuration for the robot.

6.1 Planning to a Pose Goal

The code below shows how to plan to move the robot to a pose goal using the C++ API.

```

moveit::planning_interface::MoveGroup group('`right_arm`');

geometry_msgs::Pose target_pose1;
target_pose1.orientation.w = 1.0;
target_pose1.position.x = 0.28;
target_pose1.position.y = -0.7;
target_pose1.position.z = 1.0;
group.setPoseTarget(target_pose1);

moveit::planning_interface::MoveGroup::Plan my_plan;
bool success = group.plan(my_plan);

```

The first line sets the group that the interface is working with (using the group name specified in the SRDF). Note the use of a ROS message to specify the pose target.

6.2 *Planning to a Joint Goal*

The code below shows how to plan to move the robot to a joint goal using the C++ API.

```

std::vector<double> group_variable_values;
group.getCurrentState()->copyJointGroupPositions(group.getCurrentState()
->getRobotModel()->getJointModelGroup(group.getName()),
    group_variable_values);

group_variable_values[0] = -1.0;
group.setJointValueTarget(group_variable_values);

moveit::planning_interface::MoveGroup::Plan my_plan;
success = group.plan(my_plan);

```

First, we get the current joint values for the group. Then, we modify one of the joint values to specify a new target. If the target is outside joint limits, no plan will be generated.

6.3 *Move to Joint or Pose Goals*

The process for moving to a joint or pose goal is the same as for planning to these goals. We will only use a different function call: `move()`.

```

success = group.move(my_plan);

```

6.4 Adding Objects into the Environment

Objects can be easily added using both the C++ and Python API and through the Rviz motion planning plugin. The motion planning plugin allows users to directly import .STL files (e.g. a representation of the workcell). The main parameters that can be adjusted are the collision model for the added object and the location of the object. MoveIt! allows different types of collision models including primitives (box, cylinder, sphere, cone) and mesh models. Mesh models should be simplified as far as possible to minimize the number of triangles in them.

Workcell components can also be added directly using the `PlanningSceneInterface` class.

```

moveit::planning_interface::PlanningSceneInterface
    planning_scene_interface;

moveit_msgs::CollisionObject collision_object;
collision_object.header.frame_id = group.getPlanningFrame();
collision_object.id = ``box1``;

/* Define a box to add to the world. */
shape_msgs::SolidPrimitive primitive;
primitive.type = primitive.BOX;
primitive.dimensions.resize(3);
primitive.dimensions[0] = 0.4;
primitive.dimensions[1] = 0.1;
primitive.dimensions[2] = 0.4;

geometry_msgs::Pose box_pose;
box_pose.orientation.w = 1.0;
box_pose.position.x = 0.6;
box_pose.position.y = -0.4;
box_pose.position.z = 1.2;

collision_object.primitives.push_back(primitive);
collision_object.primitive_poses.push_back(box_pose);
collision_object.operation = collision_object.ADD;

std::vector<moveit_msgs::CollisionObject> collision_objects;
collision_objects.push_back(collision_object);

planning_scene_interface.addCollisionObjects(collision_objects);

```

Attaching and detaching the collision object from the environment is also simple. It is only important to make sure that the object has already been added to the environment before attaching an object.

```

/* Attach the object */
group.attachObject(collision_object.id);

/* Detach the object */
group.detachObject(collision_object.id);

```

6.5 Helpful Hints

There are several debugging steps that a user can follow in case things don't go as planned. Here are some helpful hints for what can go wrong and how to fix it.

- **Robot won't move:** If the joint limits for the robot are not set properly, the robot may not be able to move. Check the URDF of the robot and make sure that each joint has a range of joint values to move through. Check to make sure that the maximum joint value is greater than the minimum joint value.
- **Robot won't move when I define soft limits:** If soft limits are defined for the robot in the URDF, they will be used by MoveIt!. Make sure that they are valid.
- **Motion plans are not being generated successfully:** Check that no two parts of the robot are in self-collision at all joint configurations. This can especially happen if you add new parts to the robot in the URDF but have not run the robot again through the MoveIt! Setup Assistant. If any robot parts appear red, they are in collision—run the MoveIt! Setup Assistant with the complete robot model to make sure that all collision checks that need to be disabled are labeled correctly in the SRDF.
- **GUI-based Interaction is not working properly:** Robots with 6 or more degrees of freedom do well with Rviz interaction. Robots with less than 6 DOFs are harder to interact with through the plugin.
- **The motion plans are moving into collision:** MoveIt! checks each motion plan segment for collisions at a certain discretization. If this discretization value is too large, motion segments will not be checked at a fine discretization and the resulting motions may actually pass through parts of the environment. The discretization value can be adjusted using the *longest_valid_segment* parameter in the *ompl_planning.yaml* file.
- **The motion plans are moving into collision:** If using Rviz, make sure that you have pressed the Publish Planning Scene button before planning.

6.6 Additional Resources

Additional tutorials are available on the MoveIt! website from the [tutorials page](#).

- Tutorial for the Move Group Interface ([C++](#), [Python](#))—this tutorial described the main user interface to MoveIt! for users who would like to interact through a ROS interface. It is the recommended interface for all beginners.
- Tutorial for using the kinematics API ([C++](#))—this tutorial delves deeper into the use of kinematics (forward and inverse kinematics) using the programmatic C++ API. This tutorial is only recommended for more advanced users of MoveIt!.
- Tutorial for the Planning Scene API ([C++](#), [ROS](#))—this tutorial explains the structure and interface to the planning scene API.

- Loading motion planners from plugins, using kinematic constraints: (C++)—this tutorial explains how to load and use motion planners directly from the C++ interface and also how to specify and use some types of kinematic constraints. It is intended for advanced users.
- Using the motion planning pipeline and planning request adapters: (C++)—this tutorial explains how to use the planning request adapters as part of a motion planning pipeline to change the input to and output from the motion planners. It is intended for advanced users.

7 Connecting to a Robot

MoveIt! can connect directly to a robot through a ROS interface. The requirements on a ROS interface include a source of joint information, transform information and an interface to a trajectory controller:

- **Joint States:** A source of joint state information is needed. This source must publish the state information (at least the position of each joint) at a reasonable rate on the *joint_states* topic. A typical rate is 100Hz. Different components can publish on the same topic and all the information will be combined internally by MoveIt! to maintain the right state. Note that MoveIt! can account for *mimic* joints, i.e. coupled joints where a single actuator or motor controls the motion of two joints.
- **Transform Information:** MoveIt! uses the joint state information to maintain its own transform tree internally. However, joint state information does not contain information about external virtual joints, e.g. the position of the robot in an external map. This transform information must exist on the ROS transform server using the TF package in ROS.
- **Trajectory Action Controller:** MoveIt! also requires the existence of a trajectory action controller that supports a ROS *action* interface using the FOLLOWJOINT-TRAJECTORY action in the *control_msgs* package.
- **Gripper Command Action Interface–OPTIONAL:** The gripper command action interface, allows for easy control of a gripper. It differs from the Trajectory Action interface in allowing the user to set a maximum force that can be applied by the gripper.

7.1 Configuring the Controller Interface

Configuring the controller interface requires generating the right controller configuration YAML file. An example of a controller configuration for two arms of a robot and a gripper is given below:

```
controller_list:
- name: r_arm_controller
  action_ns: follow_joint_trajectory
  type: FollowJointTrajectory
  default: true
  joints:
    - r_shoulder_pan_joint
    - r_shoulder_lift_joint
    - r_upper_arm_roll_joint
    - r_elbow_flex_joint
    - r_forearm_roll_joint
    - r_wrist_flex_joint
    - r_wrist_roll_joint
- name: l_arm_controller
  action_ns: follow_joint_trajectory
  type: FollowJointTrajectory
  default: true
  joints:
    - l_shoulder_pan_joint
    - l_shoulder_lift_joint
    - l_upper_arm_roll_joint
    - l_elbow_flex_joint
    - l_forearm_roll_joint
    - l_wrist_flex_joint
    - l_wrist_roll_joint
- name: gripper_controller
  action_ns: gripper_action
  type: GripperCommand
  default: true
  joints:
    - l_gripper_joint
    - r_gripper_joint
```

This configuration should go into a YAML file (called `controllers.yaml`) inside the MoveIt! config package. Now, create the controller launch file (call it `robot_moveit_controller_manager.launch` where *robot* is the name of your robot—the robot name needs to match the name specified when you created your MoveIt! config directory).

Add the following lines to this file:

```
<launch>
<!-- Set the param that trajectory_execution_manager needs to find the
      controller plugin -->
<arg name='moveit_controller_manager'
      default='moveit_simple_controller_manager/MoveItSimpleControllerManager' />
<param name='moveit_controller_manager' value='${arg
      moveit_controller_manager}' />
<!-- load controller_list -->
<rosparam file='${find my_robot_name_moveit_config}/config/controllers.yaml' />
</launch>
```

MAKE SURE to replace *my_robot_name_moveit_config* with the correct path for your MoveIt! config directory.

Now, you should be ready to have MoveIt! talk to your robot.

7.2 Debugging Hints

The FollowJointTrajectory or GripperCommand interfaces on your robot must be communicating in the correct namespace. In the above example, you should be able to see the following topics (using rostopic list) on your robot:

```
/r_arm_controller/follow_joint_trajectory/goal
/r_arm_controller/follow_joint_trajectory/feedback
/r_arm_controller/follow_joint_trajectory/result
/l_arm_controller/follow_joint_trajectory/goal
/l_arm_controller/follow_joint_trajectory/feedback
/l_arm_controller/follow_joint_trajectory/result
/gripper_controller/gripper_action/goal
/gripper_controller/gripper_action/feedback
/gripper_controller/gripper_action/result
```

You should also be able to see (using *rostopic info topic_name*) that the topics are published/subscribed to by the controllers on your robot and also by the *move_group* node.

7.3 Integrating 3D Perception

MoveIt! also integrated 3D perception for building an updated representation of the environment. This representation is based on the OCTOMAP package. The 3D perception elements include components that perform *self-filtering*, i.e. filter out parts of the robot from sensor streams. Input to the Octomap is from any source of 3D information in the form of a depth map. The resolution of the OCTOMAP is adjustable. Using a coarser resolution ensures that the 3D perception pipeline can run at a reasonable rate. A 10cm resolution should result in an update rate of close to 10Hz. The 3D perception can be integrated using a YAML configuratoin file.

```
sensors:
- sensor_plugin: occupancy_map_monitor/DepthImageOctomapUpdater
  image_topic: /head_mount_kinect/depth_registered/image_raw
  queue_size: 5
  near_clipping_plane_distance: 0.3
  far_clipping_plane_distance: 5.0
  shadow_threshold: 0.2
  padding_scale: 4.0
  padding_offset: 0.03
  filtered_cloud_topic: filtered_cloud
```

You will now need to update the `moveit_sensor_manager.launch` file in the launch directory of your MoveIt! configuration directory with this sensor information (this file is auto-generated by the Setup Assistant but is empty). You will need to add the following line into that file to configure the set of sensor sources for MoveIt! to use:

```
\begin{verbatim}
<rosparam command='load' file='${find
  my_moveit_config}/config/sensors_kinect.yaml' />
\end{verbatim}
```

Note that you will need to input the path to the right file you have created above. You will also need to configure the Octomap by adding the following lines into the `moveit_sensor_manager.launch`:

```
<param name='octomap_frame' type='string'
  value='odom_combined' />
<param name='octomap_resolution' type='double'
  value='0.05' />
<param name='max_range' type='double' value='5.0' />
```

7.4 Helpful Hints

- **Controller Configuration:** MoveIt! does not implement any controllers on its own. Users will have to implement their own controller or, preferably, use ROS-Control [6] as a controller framework. The ROS-Control package includes trajectory interpolation and smoothing that is essential for smooth operation on industrial and other robots.
- **Sensor Configuration:** The self-filtering routines function best if the URDF is an accurate representation of the robot. Padding parameters can also be used to adjust the size of the meshes used for self-filtering. It is always best to add a little padding to the meshes since uncertainty in the motion of the robot can cause the *self-filtering* to fail.

8 Building Applications with MoveIt!

MoveIt! is a platform for robotic manipulation. It forms an ideal base to build large-scale applications. Examples of such applications include:

- **Pick and Place:** MoveIt! includes a pick and place pipeline. The pipeline allows pick and place tasks to be fully planned given a set of grasps for an object to be picked and a set of place locations where the object can be placed. The pipeline utilizes a series of manipulation planning *stages* that are configured to run in

parallel. They include (a) freespace motion planning stage to plan the overall motion of the arm, (b) cartesian planning stages to plan approach and retreat motions, (c) inverse kinematics stages to compute arm configurations for pick and place and (d) perception and grasping interfaces for object recognition and grasp planning respectively.

- **Process Path Planning:** Complex processes like gluing, welding, etc. often require an end-effector to follow a prescribed path while avoiding collisions, singularities and other constraints. MoveIt! can plan such process paths using a cartesian planning routine built into MoveIt! itself.
- **Planning Visibility Paths:** MoveIt! includes the ability to process visibility constraints. Visibility planning is particularly useful when planning inspection paths, i.e. planning a path for inspecting a complex part with a camera mounted on the end-effector of an arm.
- **Tele-operation:** MoveIt! has been used for tele-operation in complex environments, e.g. in the DARPA Robotic Challenge. The ability to visualize complex environments while also planning collision free paths allows full teleoperation applications to be built with MoveIt!.

9 Conclusion

MoveIt! has rapidly emerged as the core ROS package for manipulation. In combination with ROS-Control, it provides a framework for building core functionality and full applications for any robotics task. The use of the MoveIt! setup assistant has made MoveIt! more accessible to new and intermediate users. It has allowed new robotic platforms to be easily integrated into ROS. The next goal for MoveIt! development is to extend its capabilities to provide even more out of the box capabilities and enable better integration with other types of sensing (e.g. force/tactile) sensing. We also aim to extend MoveIt! to whole-body manipulation tasks to enable more applications with humanoid robots. MoveIt! can also form the basis for collaborative robots, enabling the next generation of tasks where humans and robots work together. For more information, users are referred to the following resources for MoveIt!:

- [The MoveIt! Website](#)
- [The MoveIt! Tutorials](#)
- [MoveIt! High-level Concept Documentation](#)
- [MoveIt! mailing list](#)
- [MoveIt! Source code](#)

Acknowledgments MoveIt! is developed and maintained by a large community of users. Special mention should be made of Dave Hershberger, Dave Coleman, Michael Ferguson, Ioan Sucan and Acorn Pooley for supporting and maintaining MoveIt! and its associated components in ROS over the last few years.

References

1. S. Chitta, E.G. Jones, M. Ciocarlie, K. Hsiao, Perception, planning, and execution for mobile manipulation in unstructured environments. *IEEE Robot. Autom. Mag. Special Issue on Mobile Manipulation* **19**(2), 58–71 (2012)
2. S. Chitta, E.G. Jones, I. Sucan, Arm Navigation. http://wiki.ros.org/arm_navigation (2010)
3. J. Pan, S. Chitta, D. Manocha, FCL: a general purpose library for collision and proximity queries, in *IEEE International Conference on Robotics and Automation, Minneapolis, Minnesota* (2012)
4. D. Coleman, Integrating IKFast with MoveIt!: A Tutorial. http://docs.ros.org/hydro/api/moveit_ikfast/html/doc/ikfast_tutorial.html (2014)
5. S. Chitta, A. Pooley, D. Hershberger, I. Sucan, MoveIt! <http://moveit.ros.org> (2015)
6. The ROS Control Framework. http://wiki.ros.org/ros_control 2014